

Autellix: 面向通用程序式 LLM 智能体的高效服务引擎

Michael Luo^{1,2} Xiaoxiang Shi^{3,†} Colin Cai^{1,†} Tianjun Zhang¹ Justin Wong¹
Yichuan Wang¹ Chi Wang² Yanping Huang² Zhifeng Chen² Joseph E. Gonzalez¹ Ion Stoica¹

¹UC Berkeley ²Google DeepMind ³Shanghai Jiao Tong University

摘要

大语言模型 (LLM) 应用正从简单聊天机器人, 演变为动态、通用的智能体程序。这些程序增加 LLM 调用次数与输出 token 数, 使 AI 智能体能够推理、探索并解决复杂任务。然而, 现有 LLM 服务系统忽略程序与调用之间的依赖关系, 错过了大量优化机会。我们的分析发现, 提交到 LLM 服务引擎的程序会经历较长的累计等待时间, 主要原因是在单次 LLM 请求层面和程序层面都存在队头阻塞。

为解决这一问题, 我们提出 Autellix。该 LLM 服务系统将程序视为一等对象, 以降低程序的端到端延迟。Autellix 拦截程序提交的 LLM 调用, 为调度器提供程序层面的上下文。我们提出两种调度算法, 分别面向单线程程序与分布式程序: 根据所属程序此前已完成调用的信息, 抢占 LLM 调用并设置其优先级。实验表明, 在多种 LLM 与智能体工作负载上, 与 vLLM 等先进系统相比, Autellix 在相同延迟下将程序吞吐量提高至 4-15 倍。

1 引言

以大语言模型 (LLM) 为基础的自主智能体, 可以通过扩大推理计算规模来增强问题求解能力, 即增加输出 token 数或 LLM 调用次数 [9, 12, 22, 30, 31, 65]。更多调用和 token 使智能体获得更强的推理 [19, 76, 84, 85]、规划与搜索能力 [56, 91], 能够从先前经验中进行自反思 [34, 64, 87], 还可实现多个智能体协作 [20, 78, 95]。这些技术让智能体通过工具与外部环境有效交互 [54, 59, 85], 完成自主浏览网页 [27, 83, 92]、解决 GitHub issue [29, 74, 80]、证明困难数学问题 [18, 35] 等复杂任务。

推理阶段技术与智能体应用的兴起, 标志着 LLM 应用从静态、专门用途的形态 [15, 41], 转向高度动态、通用的智能体程序 [37, 78, 90]。更准确地说, 智能体程序是一个以有向无环图 (DAG) 表示的动态执行 workflow, 其中包含一个或多个智能体的 LLM 调用, 以及外部中断; 外部中断包括工具调用, 即外部 API 调用、通用代码执行或人工输入 (§2)。我们假设程序的 LLM 调用模式只有在运行时才会显现, 因此难以事先完整知道或预测整个图。¹

[†]同等且重要的贡献。

¹译注: 这里的 DAG 表示运行过程中各次调用的因果依赖; 程序

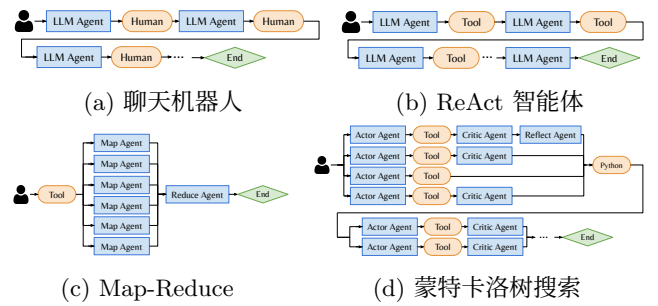


图 1: 智能体程序的执行 workflow。智能体程序是遵循有向无环图 (DAG) 的高度动态 workflow, 包括一个或多个智能体的 LLM 调用, 以及外部中断, 例如工具调用或人工交互。图中 LLM Agent 为 LLM 智能体, Human 为人工交互, Tool 为工具, End 为结束; Map Agent 与 Reduce Agent 分别为映射、归约智能体; Actor、Critic、Reflect Agent 分别为行动、评判、反思智能体; Python 为通用代码执行。

图 1 以单线程和多线程示例展示智能体程序的高度动态性。单线程程序有两个变化维度: 其一是取决于用户提示词的程序长度; 其二是由控制流决定的 LLM 调用与中断序列。例如, Chatbot 和 ReAct (推理与行动) [85] 智能体都交替执行 LLM 调用与中断, 后者来自人工输入或工具调用, 并依据人或 LLM 的决定终止 (图 1a、1b) [85]。多线程程序通常形成 DAG。经典多线程程序 Map-Reduce, 以及蒙特卡洛树搜索 (MCTS), 在执行过程中分叉和汇合的线程数都可能变化; 每条线程还可能具有不同的 LLM 调用与中断序列 (图 1c、1d)。MCTS 尤其广泛用于推理智能体和网页智能体的搜索与规划 [11, 44, 56, 91]。

vLLM 等现有 LLM 服务引擎 [36], 主要优化单次 LLM 调用或静态 LLM 应用 [41], 采用的手段包括提高键值 (KV) 缓存效率 [36, 90]、加速 CUDA 内核 [77, 94]、改进 LLM 请求调度算法 [2, 77]。但这些优化没有考虑程序层面的上下文, 例如同一个程序的 LLM 调用依赖, 或程序总执行时间等统计信息。结果是, 这些系统在复杂程序上的端到端性能通常不够理想, 尤其是程序端到端延迟 (§3)。

图 2 展示了在时刻 $t=0$, 两个长程序 A、B 与两个短

控制逻辑可以包含循环, 每次迭代展开为新的调用节点。动态、非确定的图形态并不意味着本文显式给出了分支概率或随机图分布。

程序	LLM 调用次数	每次调用的解码步数
A	4	{4,3,1,1}
B	3	{3,3,4}
C	2	{1,2}
D	1	{4}

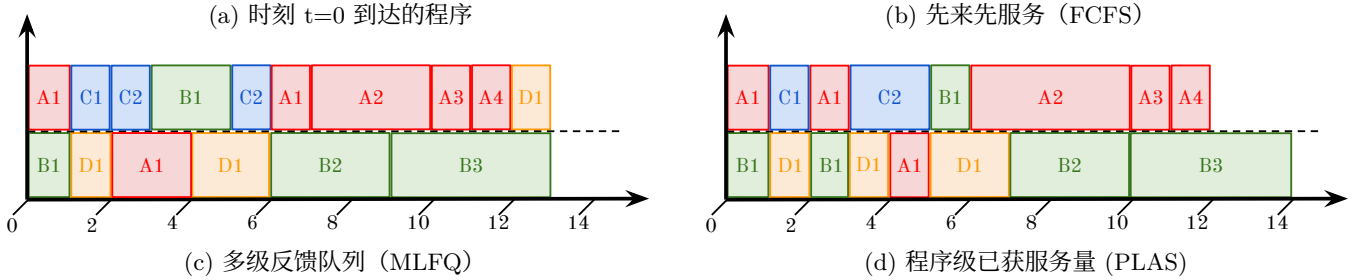


图 2: 最大批量为 2 的 LLM 服务引擎上的调用执行甘特图。纵轴表示两个批处理槽位, 横轴表示解码步骤或时间单位。(a) 四个程序的调用次数与每次解码步数不同, A、B 为长程序, C、D 为短程序。(b) FCFS 中, 长调用推迟短调用, 产生队头阻塞, 总等待时间为 18 个单位。(c) MLFQ 通过抢占缓解调用级阻塞, 但仍有程序级阻塞: A、B 的新调用进入最高优先级队列, 推迟 D, 总等待仍为 18 个单位。(d) 程序级已获服务量 (PLAS) 利用程序统计信息, 推迟 A、B 的后续调用, 优先推进 C、D, 将总等待降至 12 个单位。图中 A1、A2 等标识表示相应程序的第几次调用; 跨多个时间片重复出现同一标识, 表示同一次调用被抢占后恢复。

程序 C、D 突发提交到最大批量为 2 的 LLM 服务引擎的情形。每个程序包含一次或多次 LLM 调用, 解码长度各不相同, 如图 2a 所示。在不感知程序的先来先服务 (FCFS) 策略下, 即 vLLM 的默认策略 [36], 较长的 LLM 调用会阻止其他调用执行, 造成调用级队头 (HoL) 阻塞, 如图 2b。程序 A、B 的首次长调用先执行, 使 C、D 分别直到 $t=3, 4$ 才开始执行。反复的队头阻塞使总等待时间达到 18 个时间单位。多级反馈队列 (MLFQ) 等抢占调度 [77] 可以抢占长调用, 使短调用先执行, 缓解调用级阻塞。然而, 如果缺少程序上下文, 旧程序的后续调用会反复推迟新程序, 造成程序级队头阻塞。图 2c 中, MLFQ 成功抢占了 A、B 的长调用, 使 C、D 得以开始, 但在 $t=6-12$ 又反复优先执行 A、B 的后续调用, 推迟 D。因此, MLFQ 与 FCFS 一样, 产生 18 个时间单位的总等待时间。

我们提出 Autellix, 一个面向程序执行的 LLM 推理系统。受到操作系统进程调度的启发, 我们的核心思想是: 按照所属程序此前已完成调用的总执行时间, 设置 LLM 调用优先级。长程序近期完成的可能性较低, 因此降低其调用优先级, 让较短程序先完成。图 2d 中, 短程序 C、D 不再被长程序 A、B 的后续调用阻塞, 有效消除该示例中的队头阻塞, 将总等待时间降至 12 个时间单位。

Autellix 提出一个利用全局程序统计信息的新框架, 例如程序在引擎上的累计执行时间, 以缩短等待时间并提高引擎吞吐量。我们提出两种无需预知工作负载的调度算法: 面向单线程程序的 PLAS (程序级已获服务量), 以及面向以通用动态 DAG 表示的多线程程序的 ATLAS (自适应线程级已获服务量)。PLAS 根据调用来源程序当前累计的服务量, 即执行时间, 设置 LLM 调用优先级。作为 PLAS 的推广, ATLAS 依据同一程序所有线程中最大的累计服务时间, 设置调用优先级, 从而按程序关键路径对调用排序 [88]。除了缩短等待, ATLAS 还优先执行那些否则会阻碍程序推进的关

键 LLM 调用, 降低程序的总完成跨度 (§4)。

包含数十至数百次 LLM 调用的程序, 对服务系统提出了较高需求: 单个 LLM 引擎在 MCTS 工作负载上只能处理每秒 0.2 个程序 (§6)。因此, Autellix 还会将程序的 LLM 调用路由到多个引擎。对智能体工作负载, 我们观察到, 同一程序内的调用经常共享前缀和累计会话状态, 而跨程序调用通常只共享系统提示词 [66]。为避免重新计算程序的 KV 缓存, Autellix 考虑数据局部性: 把长调用路由至其程序所属引擎, 而将短调用负载均衡到其他引擎; 短调用的大部分输入通常是系统提示词。

我们在 vLLM 等 LLM 服务引擎之上 [36] 实现了 Autellix 原型, 并提供有状态 API, 让用户建立持久会话, 这与传统无状态 API 不同 [51]。我们使用不同 LLM 和四种代表性智能体工作负载进行评估 (§6)。相对 vLLM 等先进推理系统 [36], Autellix 将吞吐量提高至 4–15 倍。在多引擎之间, 相对标准负载均衡器, 它还将吞吐量最多提高至 1.5 倍。

本文的主要贡献如下:

- 本工作首次将智能体程序形式化为由 LLM 调用与中断构成的动态、非确定性 DAG (§2)。
- Autellix 利用程序层面的统计信息改善调度决策。其无需预知工作负载的调度器, 只需要同一程序内 LLM 调用的累计服务时间 (§4)。
- Autellix 使用简单的多引擎负载均衡策略, 在数据局部性与 KV 缓存重新计算之间取得平衡 (§4)。
- 系统易于部署, 有状态 API 能与现有编程及智能体框架顺畅集成, 并取得显著吞吐量收益 (§6)。

2 背景与相关工作

为说明 Autellix 的相关背景, 我们简要介绍新兴 AI 智能体基础设施及其应用, 将其分为 LLM 服务层 (§2.1) 与上层智能体层 (§2.2), 如图 3 所示。

2.1 LLM 服务层

LLM 推理过程。驱动聊天机器人和 AI 应用的大语言模型主要采用 Transformer 架构 [71]，其中包括 GPT、Claude 和 LLaMA 等仅含解码器的模型 [10, 28, 69, 70]。每个请求的 LLM 推理分为两个阶段：预填充阶段将输入提示词转换为中间 token 状态；解码阶段依据先前的 token 序列，自回归地逐个生成新 token。为减少计算，LLM 服务系统使用 KV 缓存保存中间 token 状态，加速后续 token 生成 [36, 86]。

LLM 服务。LLM 服务系统既负责将调用路由到不同引擎，也负责各引擎内部的调用执行（图 3）。单引擎中的近期创新，与传统操作系统的内存管理、内核优化和调度等概念相呼应 [43, 67]。vLLM 等现有方案引入虚拟内存与分页，减少 KV 缓存碎片 [36]；通过共享内存缓存跨请求前缀 [41, 90]；管理 GPU、CPU 与磁盘间的缓存层次 [62, 63, 94]。其他技术改进 GPU 内核以加速自注意力 [16]，将不同算子流水线化 [94]，以及改善张量并行或流水线并行 [39, 77]。此外，引擎还可通过更好的调度缩短响应时间，例如把预填充与解码以装箱方式共同组批 [2]，以及抢占 LLM 请求 [77]。多引擎服务系统则使用实时迁移 [68]、预填充与解码分离 [57]、前缀树构建 [66] 和跨引擎 KV 缓存迁移 [41] 等技术，满足请求 SLO 并改善尾延迟。总体上，这些技术优化的是独立 LLM 请求，相当于通用程序中的一次函数调用。Autellix 则关注程序层面的优化，尤其是调度，类似于传统操作系统在多个 CPU 核心间管理整个进程。

2.2 智能体层

智能体程序。在 LLM 推理层之上，开发者构建复杂的智能体程序，编排智能体、工具与人之间的交互（图 3）。本文关注 LLM 智能体，将其定义为一个二元组，包含说明智能体角色的系统提示词，以及 LLM 模型类别。²类似传统操作系统的进程与中断，智能体程序或者通过 LLM 调用直接与服务层交互，或者进入外部中断，即在 LLM 引擎之外花费时间。智能体可通过工具执行通用函数或外部 API，从而控制数据库、机器人或互联网等环境 [6, 54, 59, 60, 83, 93]。智能体编排框架 [50]，例如 LangChain [15, 37] 和 Autogen [78]，还为开发者提供管理程序控制流的原语，决定何时执行智能体、调用工具或请求人工输入。这些原语遵循通用编程语义，包括条件语句、循环、错误处理和终止条件 [32, 37, 78, 90]。

程序还维护智能体、工具与人工输出的全局历史 [37, 41, 53, 81]。例如，LLM 聊天机器人会累计智能体输出与人工输入之间的消息 [48]。Autellix 不修改程序层，而是在程序运行时动态构建其执行图（DAG）的内部状态，将其存入进程表 (§5)。

智能体应用。除普通聊天机器人（图 1a）外，智能体应用，即程序的具体实例，可自动完成或辅助完成复杂任务，包括网页或用户界面导航，例如 OpenAI Operator [6, 27, 47, 92]；解决 GitHub issue [29, 74, 80]；求解 IMO

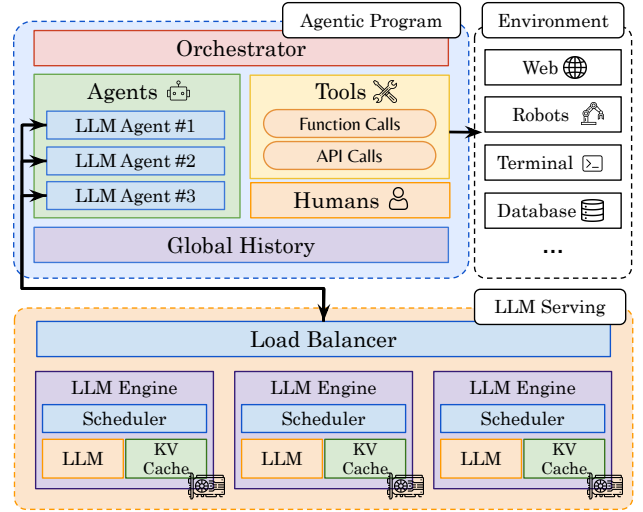


图 3: AI 智能体基础设施。上部：开发者与用户构建并执行智能体程序，编排智能体、工具和人的操作，并保存不断累积的全局历史。下部：LLM 服务系统处理调用，将其路由到一个或多个引擎。图中文字对应：Agentic Program / Orchestrator 为智能体程序/编排器，Agents 为智能体，Tools 为工具，Function Calls / API Calls 为函数 / API 调用，Humans 为人，Global History 为全局历史；Environment 为环境，Web / Robots / Terminal / Database 为网页 / 机器人 / 终端 / 数据库；LLM Serving / Load Balancer / LLM Engine / Scheduler / KV Cache 为 LLM 服务 / 负载均衡器 / 引擎 / 调度器 / KV 缓存。

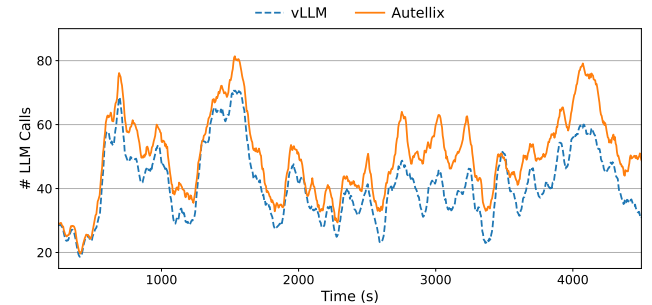


图 4: 一小时稳态运行中，服务引擎内的 LLM 调用数。缩短程序等待时间，会增加稳态下的在途调用量。横轴 Time 为时间（秒），纵轴为引擎中的调用数；两条曲线分别为 vLLM 与 Autellix。

难度的问题 [18, 35]；对多个来源的陈述进行事实核查与摘要（图 1c）[23, 41]；以及精确控制机器人 [58]。许多应用通过增加 LLM 调用次数及相应总解码 token 数，扩大推理计算规模，改善复杂任务性能。这些测试阶段方法包括：逐步推理以分解任务 [55, 76]；显式注入思路来引导推理 [85]；通过规划或搜索探索候选解 [8, 84, 91]；自我批评以评估行动 [42, 89]；自反思以从失败中学习 [34, 64]；以及多智能体协作 [21, 78]。单线程 ReAct 智能体结合思维链（CoT）技术，有效地在环境中行动（图 1b）；近期，它与 DeepSeek 式或 o1 式 LLM 结合，实现自动推理与工具调用 [19, 74, 75]。多线程 MCTS 程序 [91] 则结合同步规划、自我批评、自反思和多智能体协作（图 1d）。

²系统提示词相同、但模型不同的 LLM 智能体，例如 LLaMA [69] 与 Mistral [28]，被视为不同智能体 [73]。

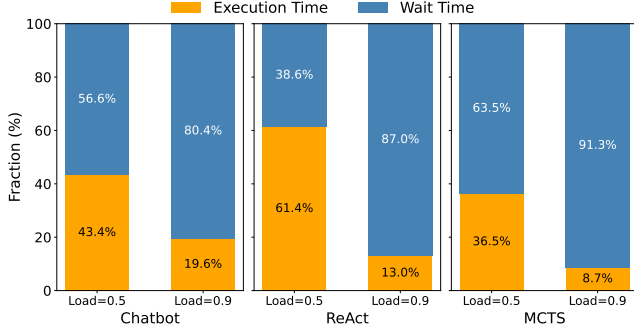


图 5: 不同程序与系统负载下的执行时间和等待时间。中等负载下, 程序已可能花费大部分时间等待, 具体比例取决于工作负载。纵轴 Fraction 为百分比; 橙色为执行时间, 蓝色阴影为等待时间; 横轴为 Chatbot、ReAct、MCTS, 各自比较 Load=0.5 与 Load=0.9 的负载设置。

此外, 分布式程序还可采用 best-of-N 采样、束搜索、前瞻方法和遗传算法, 探索并发现更优解 [13, 17, 38, 65]。LLM 具有概率性, 而上述多样的推理阶段方法, 使智能体程序及其应用呈现三种性质: (1) 动态性, 同一程序在不同用户提示词下可能产生完全不同的执行模式; (2) 非确定性, 未来行为未知, 例如程序何时决定终止; (3) 分布式性, 许多程序采用并行调用。因此, Autellix 无需预知程序, 不依赖工作负载或执行图的任何先验知识。

3 研究动机

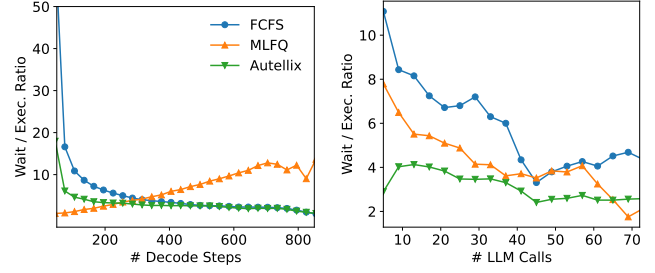
当前 AI 智能体基础设施将 LLM 服务系统与智能体程序分离 (§2)。组织从提供单次 LLM 查询转向提供上层 AI 应用时, 引擎需要优化程序层面的目标, 例如响应时间或端到端延迟 [41]。形式上, 单线程程序的端到端延迟包含三部分: (1) 等待时间, 程序所有 LLM 调用在引擎上的总排队时间; (2) 执行时间, 调用的累计前向计算时间; (3) 外部中断时间, 等待工具调用或人工输入等外部中断的时间。第三部分与 LLM 服务无关, 因此本节分析减少等待时间 (§3.1) 和执行时间 (§3.2) 的问题与机会, 随后在 Autellix 调度策略设计中解决 (§4)。

3.1 程序级等待时间

图 5 表明, 在从普通聊天机器人到 ReAct、MCTS 的各类智能体工作负载中, 随着负载增加, 程序大部分时间都用于等待。因此, Autellix 优先缩短等待时间, 既改善程序延迟, 也提高 LLM 引擎吞吐量。调用更快完成后, 程序会更快发起后续调用, 提高调用到达率。图 4 展示一小时轨迹中的稳态行为: 使用单张 A100-80GB GPU 上的 LLaMA-3.1-8B [24], 处理完整聊天会话 [1]。相对 vLLM 的 FCFS 策略, Autellix 稳定地多处理约 10 个并发 LLM 调用, 为组批提供更多机会, 从而提高吞吐量。

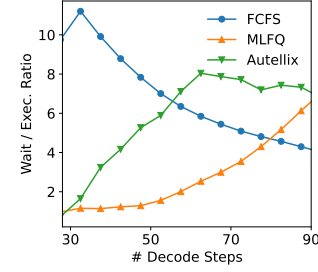
调用级阻塞。第一个挑战是 LLM 调用级队头阻塞。解码较长的调用会推迟较短调用, 造成显著等待 [77]。这一问题在 vLLM 等服务引擎中很明显 [36]: 它们等待正在进行的调用完成解码, 才调度新调用。我们评估的工作负载, 其解码步数呈长尾分布, 因此队头阻塞尤为严重(图 11)。

为衡量阻塞, 图 6 以输出 token 数为横轴, 给出 Chat-

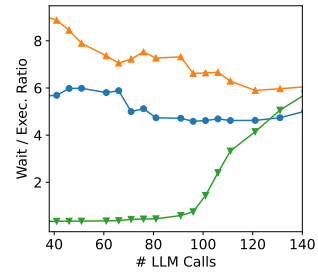


(a) Chatbot: LLM 调用

(b) Chatbot: 程序



(c) MCTS: LLM 调用



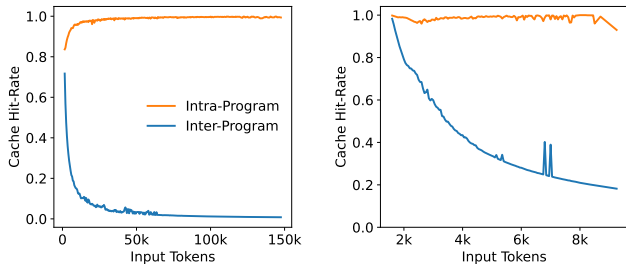
(d) MCTS: 程序

图 6: LLM 调用与程序的等待时间/执行时间之比。当短调用与短程序的等待时间远长于其执行时间时, 便出现队头阻塞。所有子图纵轴均为该比值; 调用图横轴为解码步数, 程序图横轴为 LLM 调用次数。图例为 FCFS、MLFQ 和 Autellix。

bot 与 MCTS 工作负载中 LLM 请求等待时间与执行时间之比。FCFS 下, 队头阻塞增加短调用的等待, 使该比值升高。抢占调度类似操作系统中断长时间运行的进程, 它优待短调用, 从而缓解队头阻塞。图 6 显示, MLFQ 这种抢占算法使短解码调用的比值更小。不过, 没有程序统计信息的抢占调度, 可能仍无法完全解决问题, 原因如下。

程序级阻塞。第二个挑战是程序级队头阻塞: 包含许多 LLM 请求的长程序推迟短程序。现有 LLM 调度器不感知程序, 只调度独立请求, 没有考虑调用在整个程序中的位置, 因此可能作出次优决策。实验中, 每个程序的 LLM 调用次数呈长尾分布, 进一步加重程序级阻塞 (§6)。

为量化程序级阻塞, 图 6 根据 LLM 调用次数, 给出程序等待时间与执行时间之比。两种工作负载中, 当调用次数较少时, FCFS 与 MLFQ 都产生较大比值, 说明短程序等待了较长时间。因此, MLFQ 等抢占策略的表现可能接近 FCFS, 甚至比它更差 (§6)。缺少程序上下文时, MLFQ 盲目优先执行新请求; 当长程序的新



(a) 单线程: Chatbot

(b) 多线程: MCTS

图 7: 同一程序内部与不同程序之间的前缀缓存命中率。同一程序的调用通常共享 KV 缓存, 跨程序调用通常较少共享。横轴为输入 token 数, 纵轴为缓存命中率; Intra-Program 为程序内共享, Inter-Program 为跨程序共享。

调用得到优待时, 短程序便可能饥饿。

3.2 程序级执行时间

程序执行时间主要取决于 LLM 引擎管理预填充与解码的效率。智能体工作负载经常包含较长、不断累积的预填充输入, 因此 Autellix 侧重优化预填充性能。前缀缓存可消除大量预填充计算: 它在请求间保存并复用相关 KV 缓存条目, 例如系统提示词的缓存 [41, 90]。

数据局部性。图 7 以输入长度为横轴, 给出平均缓存命中率。对于一个新到达的调用, 命中率定义为其输入 token 中, 已经在引擎 KV 缓存中完成预计算的比例。在同一程序内部, 所有输入长度对应的命中率均保持在 90% 以上, 说明该程序的调用共享相同上下文。相比之下, 跨程序命中率随输入长度呈指数下降, 表明不同程序主要只共享系统提示词。因此, 多引擎 LLM 服务系统应考虑程序的数据局部性, 因为后续调用可以复用该程序大部分 KV 缓存。

4 Autellix 的设计

本节先介绍 Autellix 的整体架构 (§4.1), 再说明两个关键组件: (1) 降低调用级和程序级阻塞的程序感知调度器 (§4.2); (2) 感知数据局部性的负载均衡器 (§4.3)。

4.1 概述

Autellix 是面向智能体程序的上层服务引擎。它有三个主要目标: (1) 为用户降低程序整体端到端延迟; (2) 为服务提供方提高 GPU 利用率; (3) 缓解程序饥饿, 改善公平性, 并以第 95、99 百分位延迟进行衡量。

假设。Autellix 无需预知程序到达、实际执行的工作流结构或一般工作负载分布。程序到达时, 其执行 DAG 尚未知; 系统在运行过程中动态构建内部表示 (IR)。因此, 它能适用于任何调用底层 LLM 引擎的程序。已有工作 [41] 把静态 LLM 应用提交到引擎, 而 Autellix 假设用户在本地机器运行通用 Python 程序, 程序调用 Autellix 后端 (§5)。

架构。图 8 展示整体架构。现有引擎通常将 LLM 调用视为无状态请求, Autellix 则是有状态的: 用户在本地执行程序, 与系统建立会话, 再陆续发起附带会话 ID

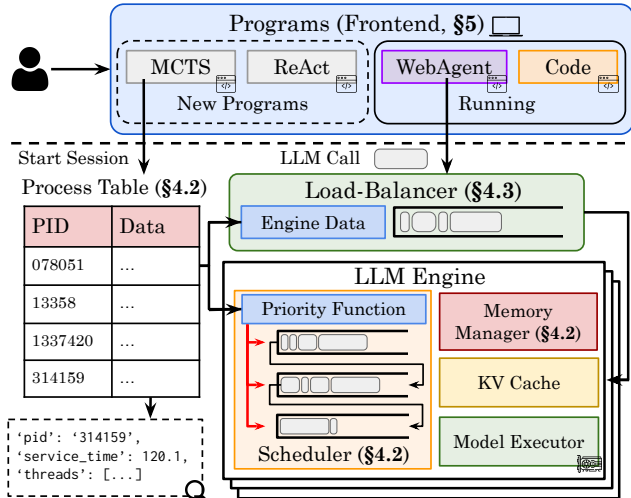


图 8: Autellix 系统架构。用户本地运行程序, 建立有状态会话并向后端提交调用; 系统通过全局进程表跟踪会话, 指导负载均衡器与调度器。图中 Programs / Frontend 为程序 / 前端, New Programs / Running 为新程序 / 运行中程序, WebAgent / Code 为网页智能体 / 代码执行; Start Session / LLM Call 为启动会话 / LLM 调用; Process Table / PID / Data 为进程表 / 程序 ID / 数据; Load-Balancer / Engine Data 为负载均衡器 / 引擎数据; Priority Function / Memory Manager / Scheduler / KV Cache / Model Executor 为优先级函数 / 内存管理器 / 调度器 / KV 缓存 / 模型执行器。示例记录中的 pid、service_time、threads 分别表示程序 ID、服务时间和线程元数据。

的调用。底层实现见第 5 节。会话开始时, 系统在全局进程表中加入对应条目 (§4.2)。该表记录程序元数据, 包括总服务时间、线程级元数据及各调用的等待时间。引擎级调度器 (§4.2) 与有状态负载均衡器 (§4.3) 都利用它: 前者决定下一解码批次的调用, 后者按程序数据局部性将调用路由到引擎。

4.2 程序感知调度器

我们提出通用、高效的调度器, 在没有先验知识的条件下缩短程序响应时间, 即端到端延迟。为缓解程序级与调用级队头阻塞, Autellix 根据程序统计信息, 例如累计总运行时间 (§4.2.1), 设置调用优先级, 并动态抢占调用 (§4.2.2)。完整调度算法见算法 1。

4.2.1 程序级优先级设置

为有效设置程序级优先级, Autellix 使用全局进程表记录关键指标, 为单线程和多线程程序的调度提供信息。

进程表。受传统操作系统启发, Autellix 维护全局进程表, 记录全部运行程序的状态。新程序到达时添加条目, 完成时删除条目。每个条目记录以下指标:

- **服务时间:** 对单线程程序, 为已完成调用在 LLM 引擎模型执行器上的累计执行时间; 对多线程程序, 为已观测到的最长关键路径执行时间。
- **等待时间:** 在 LLM 引擎调度队列中花费的时间, 用于防止饥饿。
- **引擎 ID:** 程序当前所在的一个或多个引擎, 供

算法 1 Autellix 的程序感知调度器

```

1: 过程 UPDATE_PROCESS_TABLE(调用  $c$ , 表  $pt$ )
2:    $pd = pt[c.pid]$ 
3:   // 总服务时间(PLAS),最大关键路径服务量(ATLAS)
4:    $pd.service = \max(pd.service, c.service + c.model\_time)$ 
5:   // 更新其他指标.....
6:   ...
7: 结束 过程
8: 过程 SCHEDULER(队列  $Q_1, \dots, Q_K$ , 表  $pt$ )
9:   对  $c \in C_{arrived}$  执行  $\triangleright$  新到达的 LLM 调用
10:    // 根据程序 ID 读取优先级
11:     $c.service = pt[c.pid].service$ 
12:     $c.q\_idx = i, s.t. Q_i^{low} \leq c.service \leq Q_i^{hi}$ 
13:     $Q_{c.q\_idx}.append(c), c.quanta = Q_{c.q\_idx}.quanta$ 
14: 结束 对
15: 对  $c \in \{Q_1, Q_2, \dots, Q_K\}$  执行
16:   若  $c.finished()$  则  $\triangleright$  已完成调用更新进程表
17:     UPDATE_PROCESS_TABLE( $c, pt$ )
18:      $Q_{c.q\_idx}.remove(c)$ 
19:   结束 若
20:   若  $c.quanta \leq 0$  则  $\triangleright$  降低调用优先级
21:      $Q_{c.q\_idx}.remove(c), Q_{c.q\_idx+1}.append(c)$ 
22:      $c.q\_idx = c.q\_idx + 1, c.quanta = Q_{c.q\_idx}.quanta$ 
23:   结束 若
24:    $wait = pt[c.pid].wait + c.wait$ 
25:    $service = pt[c.pid].service + c.model\_time$ 
26:   若  $wait/service \geq \beta$  则  $\triangleright$  防止饥饿
27:      $Q_{c.q\_idx}.remove(c), Q_1.append(c)$ 
28:     // 重置等待时间与模型执行时间
29:      $c.wait = 0, c.model\_time = 0$ 
30:   结束 若
31: 结束 对
32:  $B_{out} = []$   $\triangleright$  调度下一批 LLM 调用
33: 对  $c \in \{Q_1, Q_2, \dots, Q_K\}$  执行
34:   若  $engine.can\_fit(c)$  则
35:      $B_{out}.append(c)$ 
36:   否则
37:     跳出循环
38:   结束 若
39: 结束 对
40: 结束 过程

```

Autellix 负载均衡器使用 (§ 4.3)。

- **线程元数据:** 每条线程对应一个活动 LLM 调用, 因此记录程序的活动调用及各自到达、等待和服务时间。
- **最近调用到达时刻:** 该程序最近一次新调用的到达时刻, 用于识别长时间无活动的程序。
- **最近调用完成时刻:** 该程序最近一次调用的完成时刻, 用于识别较长的外部中断。

程序的一次 LLM 调用完成后, 表中相应信息得到更新。利用进程表, 调度器可以了解每个程序的全局状态, 据此安排调用。

单线程程序。 最短作业优先 (SJF) 和最短剩余处理时间 (SRPT) 等策略, 可在相应单服务器或多服务器调度设定下最优地降低响应时间 [4, 25]。不过, 它们要

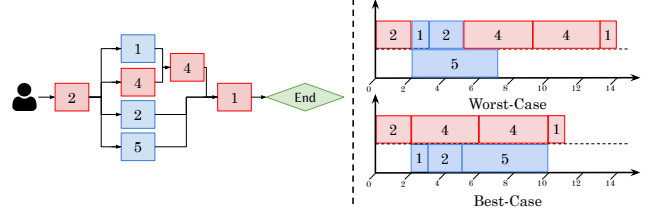


图 9: 多线程程序的关键路径。左图以红色表示 DAG 中的关键路径, 节点数值表示各调用所需时间, End 为结束。右图比较两种执行次序的完成跨度。原图注写为“最好情形 14 个单位, 最坏情形 11 个单位”; 译注: 这两个数值在原图注中颠倒了, 图形与正文一致表明 Worst-Case (最坏) 为 14, Best-Case (最好) 为 11 个单位。

求准确知道程序运行时长, 不符合 Autellix 无需预知的假设。最少已获服务量 (LAS) 算法 [45] 则提供实用替代方法, 广泛用于缺少任务信息的场景, 例如数据中心网络 [7, 14] 和深度学习集群 [26]。³

我们提出程序级已获服务量 (PLAS), 将 LAS 扩展到程序。单线程程序的服务时间, 是此前全部已完成 LLM 调用的总运行时间。形式上, 当第 j 次调用 c_j 提交, 程序 ID 为 $c_j.id$ 时, PLAS 根据此前同 ID 调用的全部运行时间 t_k 之和, 为 c_j 赋予优先级 $p(c_j)$:

$$p(c_j) = \sum_{\substack{k < j \\ c_k.id = c_j.id}} t_k \quad (1)$$

数值越大, 优先级越低。为减少计算, 调度器直接从进程表读取程序总服务时间 (第 11 行); 调用完成时, 更新所属程序总服务时间 (第 4 行)。因此, PLAS 自然优待累计服务较少的程序, 帮助较短程序先完成, 缩短响应时间。⁴

多线程程序。多线程程序表示为由 LLM 调用构成的动态 DAG。程序完成时间受到 DAG 关键路径限制, 即从开始到结束的最长依赖调用链, 如图 9。无论引擎能处理多少并行调用, 关键路径上的调用全部完成后, 程序才可能终止。忽略关键路径还会导致次优完成时间: 图 9 中, DAG 的完成跨度从 11 个时间单位增加到了 14 个时间单位。

为此, 我们提出自适应线程级已获服务量 (ATLAS), 作为 PLAS 的实用推广, 依据调用沿所属程序关键路径已获得的服务时间设置优先级。ATLAS 的目标是: 为新到达的调用 c_j 设置优先级 $p(c_j)$, 依据是同一程序中其父调用 $p(c_j)$ 的优先级与已完成服务时间:

$$p(c_j) = \begin{cases} 0 & \text{if } c_j \text{ is root} \\ \max_{c_k \in \mathcal{P}(c_j)} \{p(c_k) + t_k\} & \text{otherwise} \end{cases} \quad (2)$$

其中, t_k 是父调用 c_k 的执行时间。递归组合父调用的优先级与运行时间后, $p(c_j)$ 估计到达 c_j 的最长累计服务链, 为无需预知未来的关键路径估计提供依据。公式

³译注: 调度最优性结论依赖具体假设, 如服务器数量、到达模型、是否允许抢占及目标函数; 不能将这里的概括直接推广为一般多服务器动态 DAG 的最优性保证。

⁴译注: 公式中的程序 ID 比较写作 $c_{k.id} = c_{j.id}$, 但正文定义的是当前调用 c_j 。原公式保留, 该下标不一致应视为排版笔误; 其意图是累加同一程序此前已完成调用的时间。

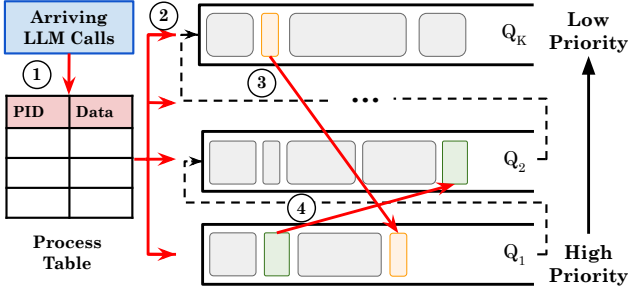


图 10: 按离散化优先级调度的 LLM 调用生命周期。图中 Arriving LLM Calls 为新到达调用, Process Table 为进程表, PID / Data 为程序 ID / 数据; 从 High Priority 到 Low Priority 为优先级从高到低。步骤 ① 查询程序服务量, 分入对应队列, 耗尽时间片后降级, 等待/服务比过大时提升到最高优先级队列。

中, 根调用的初始优先级为零, 其他调用取各父调用优先级与执行时间之和的最大值。

不过, 同时做到优待短程序与优先推进最长的关键路径线程, 并不容易。为此, *ATLAS* 在每个程序的进程表条目中, 只保存一个标量: 最长已观测关键路径。程序的每个活动调用, 都继承该值作为初始优先级; 调用完成时, 仅在它自身对应的关键路径更长时, 更新这个标量 (第 4 行)。这种简单机制不需要逐条跟踪调用依赖, 就能持续更新程序关键路径估计。因此, *ATLAS* 优待关键路径较短的程序与调用, 近似实现面向动态 DAG 的 LAS 策略。同一程序的全部调用从同一条目取得优先级, 调度器因而自然地吧并行调用归组, 防止落后的线程推迟程序完成。⁵

4.2.2 抢占调度

Autellix 根据所属程序历史设置调用优先级。不过, 按连续优先级调度与抢占, 最坏可能退化为轮转调度 [14], 比 FCFS 更差, 还引入不必要的上下文切换, 包括频繁在 CPU、GPU 间换入换出 KV 缓存 [77]。因此, 系统把优先级离散为有限队列, 类似操作系统中的 MLFQ [5, 14, 26]。

基于程序的多级调度。 Autellix 将调用优先级离散到 K 个队列 ($Q_1, Q_2, \dots, Q_K$), 优先级从 Q_1 至 Q_K 逐渐降低。每个队列 Q_i 覆盖区间 $[Q_i^l, Q_i^h]$, 满足 $Q_1^l = 0$ 、 $Q_K^h = \infty$ 以及 $Q_{i+1}^l = Q_i^h$ 。

图 10 中, 新调用到达时, 系统从进程表查询所属程序的优先级 $p(c)$ (①, 第 11 行)。传统 MLFQ 让所有新调用从最高优先级队列 Q_1 开始, 而这里按照离散化优先级 $p(c) \in [Q_i^l, Q_i^h]$, 把调用分到第 i 个队列 (②, 第 12 行)。随后调用得到该队列的时间片, 并按队列内 FCFS 顺序执行 (第 13、35 行)。调用耗尽时间片后, 降级到更低优先级队列 (③, 第 20–23 行)。如果等待过久, 系统使用下述防饥饿机制 (④, 第 24–30 行)。调用完成解码后, 更新进程表 (第 16–18 行)。

⁵译注: 递推公式描述了依据真实父节点的路径服务量, 而实现采用程序级单一标量, 在调用到达时继承、完成时取最大值; 两者的状态信息精度不同。该标量是已执行历史的近似, 不是未知未来 DAG 的精确关键路径, 也不估计分支概率或剩余时间分布。

算法 2 Autellix 的负载均衡器

```

1: 过程 LOAD_BALANCER(调用  $c$ , 表  $pt$ , 列表 Engines)
2:   若  $LEN(c.tokens) \leq 2048$  则  $\triangleright$  短请求
3:      $assigned\_engine = LEAST\_USED(Engines)$ 
4:   否则
5:     若  $c.pid \in pt$  则  $\triangleright$  程序已绑定引擎
6:        $assigned\_engine = pt[c.pid]$ 
7:     否则
8:       // 选择利用率最低的引擎
9:        $assigned\_engine = LEAST\_USED(Engines)$ 
10:       $pt[c.pid] = assigned\_engine$ 
11:   结束 若
12: 结束 若
13: 返回  $assigned\_engine$ 
14: 结束 过程
15: 过程 LEAST_USED(列表 Engines)
16:   // 并行查询各引擎的工作负载
17:    $workloads = QUERY\_ENGINE\_WORKLOADS(Engines)$ 
18:    $least\_used\_engine = ARGMIN(workloads)$ 
19:   返回  $least\_used\_engine$ 
20: 结束 过程

```

防止饥饿。 离散优先级或 MLFQ 式算法, 会使长时间、低优先级程序饥饿 [14, 26, 77]。简单地提升等待超过阈值的调用, 会把 Autellix 变回朴素 MLFQ: 长程序调用一旦进入 Q_1 , 又会打断短程序 [5, 77]。因此, 我们还利用进程表衡量程序级饥饿。具体而言, 对程序 p , 如果总等待时间 ($W_{total} = W_p + W_c$) 与服务时间 ($T_{total} = T_p + T_c$) 之比超过阈值 β , 便将调用 c 提升到 Q_1 :

$$\frac{W_{total}}{T_{total}} \geq \beta$$

调节 β , 可在平均程序响应时间与公平性之间权衡。提升后只将 W_c 和 T_c , 即当前调用的等待与运行时间清零, 使同一程序的线程或活动调用更可能一起得到提升 (第 29 行)。

内存管理。 抢占调度使引擎需要处理大量并发调用, KV 缓存块不断换入换出, 以服务不同请求, 造成频繁 GPU–CPU 传输 [77]。已有工作在处理当前调用时, 提前换入下一轮调用的 KV 缓存, 以降低开销 [77]。不过, Autellix 采用同步机制, 需要实时更新每个调用的时间片与进程表。它因此采用两项优化, 分别减少换入换出次数与单次开销。

第一, 使用多步调度, 每 N 个解码步骤调度一次, 而不是每步都调度。由于部分请求可能提前完成, 调度器会额外准备已位于 GPU 的排队请求; 若请求在 N 步之前完成, 就可立即补入新请求。第二, 采用更高效的 GPU–CPU 交换内核: 不再对每个块分别发起异步传输, 而是将所有 KV 块收集到连续缓冲区, 一次传输。这样减少碎片与逐块开销, 提高 PCIe 带宽利用率, 降低端到端交换延迟 (§5)。

4.3 负载均衡器

随着智能体工作负载增长, 必须部署多个引擎副本。但分发请求时忽略数据局部性, 会导致次优性能 [66]。

对智能体工作负载的分析 (§3.2) 揭示，短请求与长请求存在重要区别。少于 2048 个 token 的短请求，由于共享系统提示词，在任意引擎上都能获得较高命中率 ($\geq 75\%$)。强制这些请求保持局部性收益很小，还可能在大型并行程序集中于某些引擎时造成负载不均。因此，把短请求简单地分到负载最低的引擎，就能以较少开销保持性能。长请求则更依赖所属程序的数据局部性：它们与该程序具有大量前缀重叠，持续路由到同一引擎能显著减少重新计算，因此可以接受偶尔的排队延迟。

已有工作依赖复杂前缀树来衡量数据局部性 [66]；我们采用简单方法，将短请求动态路由至负载最低的引擎，而将长请求固定到所属程序对应的引擎。算法 2 形式化描述这一方法。实验表明，Autellix 的负载均衡器能改善异构工作负载上的吞吐量和延迟 (§6)。

5 实现

Autellix 是多引擎 LLM 推理服务系统，由前端、调度器和负载均衡器组成，共约 5000 行 Python 与 CUDA/C++ 代码。

前端。 Autellix 扩展 OpenAI Chat Completion 和 vLLM Python API [36, 49]，提供有状态接口，但开发者使用起来仍类似无状态接口。用户只需在 Python 应用中导入库；程序初始化时，系统自动向后端发出 `start_session` 请求，取得唯一会话标识符，并在进程表中创建条目。后续 LLM 调用在发往后端前，透明地附加正确的会话、程序和线程 ID。程序完成或发生错误时，系统调用 `end_session`，删除相应条目。作为研究原型，当前前端没有防止用户修改底层包的保护措施，这一限制留待未来解决。

LLM 引擎。 Autellix 基于 vLLM v0.6.1 [36]。为使修改集中，我们只改调度器，加入 *PLAS*、*ATLAS* 和 *MLFQ* 策略，以及提高效率的内存交换内核。这便于实验，也使性能收益的来源更明确。调度器遵循前节算法 (§4)。我们还发现，vLLM 通过 `cudaMemcpyAsync` 逐个传输 KV 块，形成碎片化的传输，无法充分利用 PCIe 带宽，并带来重复 DMA 设置等开销。因此，我们分配主机缓冲区，将全部 KV 块合并为一个连续块，进行一次批量传输。结果见下节 (§6)。

多引擎。 论文所使用的 vLLM 尚不能同时管理多个 LLM 引擎。为评估负载均衡策略，我们在 `AsyncLLMEngine` 之上构建 `AsyncMultiLLMEngine`。每个副本运行于独立 Python 进程，由上层协调引擎通过 `mp.Queue` 与 `mp.Pipe` 等标准进程间通信 (IPC) 原语管理。收到请求后，协调引擎将其分给合适副本，并立即向前端返回类似 `future` 的对象，无需阻塞。选定进程异步执行任务，通过 IPC 发送完成结果；协调引擎收到结果后解析 `future`，将输出交给前端。这样可以并行处理多个请求，协调引擎负责非阻塞的路由、资源分配与结果收集。

6 评估

本节分析代表性智能体工作负载，将 Autellix 与先进 LLM 服务系统比较，并通过消融实验评估设计选择。

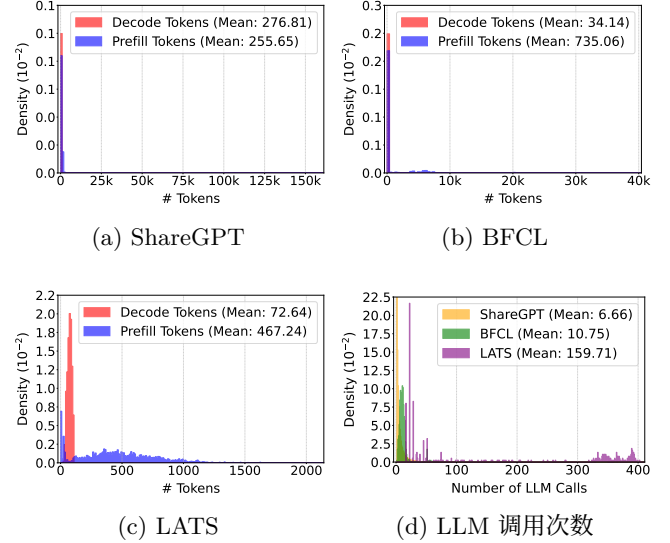


图 11: 工作负载分析。各类程序的调用统计：(a) ShareGPT、(b) BFCL、(c) LATS 的输入与输出长度分布；(d) 每个工作负载的调用次数分布。前三图横轴为 token 数，Decode Tokens / Prefill Tokens 为解码 / 预填充 token，Mean 为均值；末图横轴为 LLM 调用次数，图例标注各类均值。纵轴 Density 为概率密度，采用图示的十的负二次方标度。

6.1 工作负载

真实工作负载实验使用四类代表性智能体负载，解码 token 数、预填充 token 数与调用次数差异很大 (图 11)。

聊天机器人: ShareGPT [1]. ShareGPT 包含用户生成的会话输入与输出，代表常见聊天应用。调用次数呈长尾分布，平均为 6.66 次，最多 80 次 (图 11d)。其会话特征体现为较重的解码负载：每次调用平均生成 277 个 token，而预填充平均只有 256 个 token，即短提示词产生详细回答 (图 11a)。实验把整个会话作为一个程序重放，而非只重放第一轮。

ReAct 智能体: BFCL [79]. Berkeley Function Calling Leaderboard (BFCLv3) 评估 LLM 在多轮、多步骤工具使用任务中的能力。相比 ShareGPT，BFCL 的调用次数长尾较弱，每个程序平均 10.75 次调用，最多 70 次 (图 11d)。它的预填充负载较重，较长系统提示词与详细工具签名使每次调用平均预填充 735.06 个 token；解码较短，平均 34.14 个 token (图 11b)。因此，BFCL 代表在大量预填充、短解码与函数调用间交替的动态 workflow。

蒙特卡洛树搜索: LATS [91]. LATS 工作负载来自在 HotpotQA [82] 上运行 MCTS，计算量较大，涉及大量并行调用。每个程序平均包含 159.7 次 LLM 调用，比 ShareGPT 或 BFCL 高约一个数量级 (图 11d)。每次调用平均预填充 467.2 个 token、解码 72.6 个 token (图 11c)。这些分布反映 MCTS 反复迭代与并行的特性，要求服务系统高效处理大量并发调用。

混合负载。 我们以相同概率从以上三类负载采样，形成多样的混合负载，用于压力测试系统处理不同程序类型的能力。

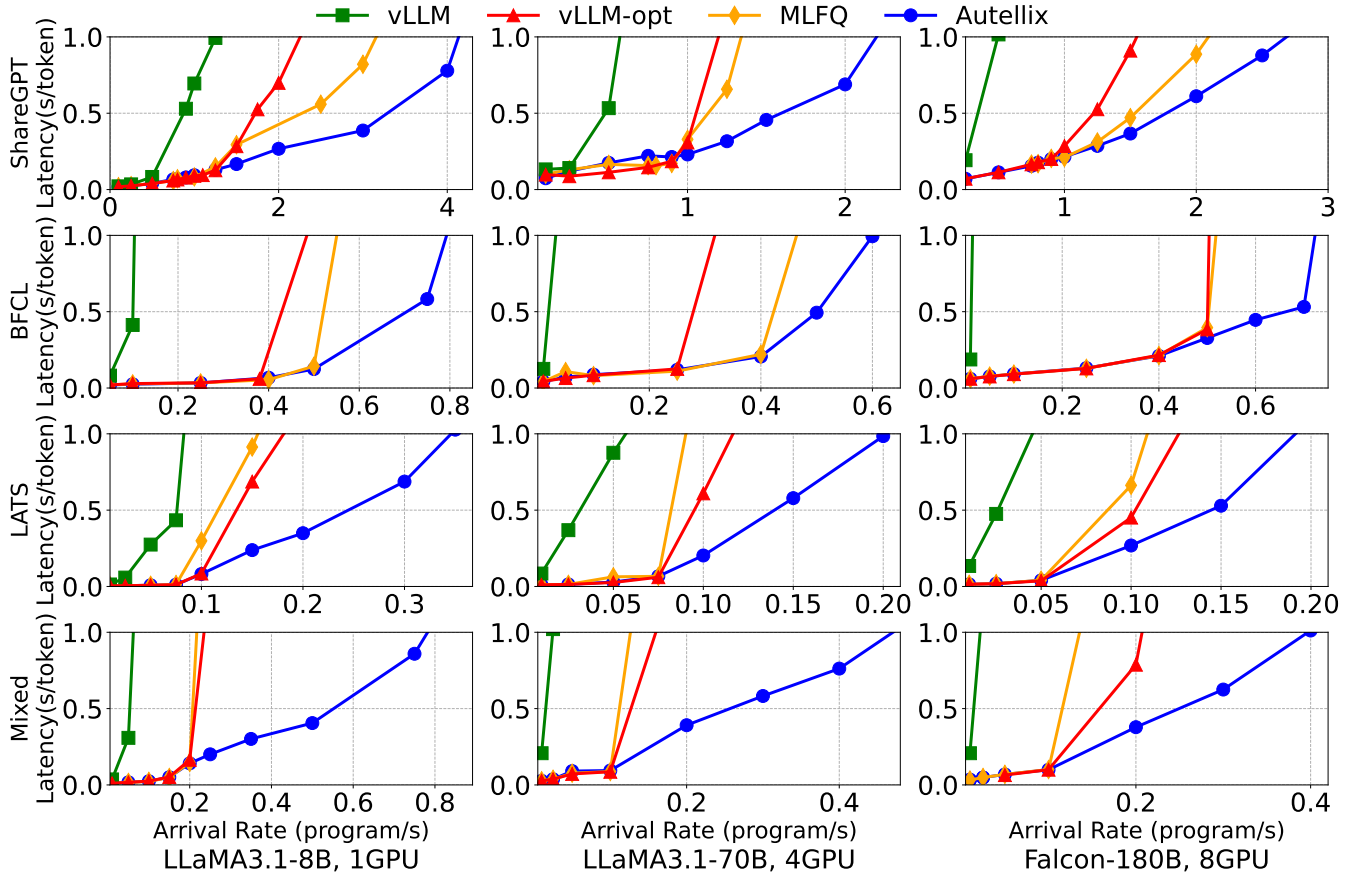


图 12: 单引擎主要结果。四类代表性工作负载上, 不同服务系统的平均延迟。各行依次为 ShareGPT、BFCL、LATS 和混合负载; 各列依次为单 GPU 的 LLaMA-3.1-8B、四 GPU 的 LLaMA-3.1-70B、八 GPU 的 Falcon-180B。横轴 Arrival Rate 为程序到达率 (程序/秒); 纵轴 Latency 为程序级 token 延迟 (秒/token), 并非原始端到端总时间。图例为 vLLM、vLLM-opt、MLFQ 和 Autellix。

实验随机采样上述工作负载中的程序, 而非独立 LLM 调用, 并使用泊松过程 λ 生成程序到达, 遵循已有方法 [36, 77]。这种设置用于接近真实场景。⁶

6.2 实验设置

模型与平台。评估使用 LLaMA-3.1-8B、LLaMA-3.1-70B 和 Falcon-180B, 分别运行于 1、4、8 张 GPU。平台为 GCP Compute Engine 的 a2-ultragpu-8g 实例, 配置八张通过 NVLink 连接的 A100-SXM4-80GB GPU、1360 GB 主机内存、PCIe-4.0×16, 以及 2 TB 磁盘。

指标。现有服务系统主要使用请求级指标, 例如首 token 时间 (原文缩写 TFFT) 与每输出 token 时间 (TPOT), 后者也称 token 延迟 [36, 77, 94]。这些指标没有反映智能体程序端到端延迟。我们因此定义程序级 token 延迟: 程序总响应时间除以生成的 token 数。⁷高吞吐量程序服务系统应在高请求率下保持低程序级延迟。为简洁起见, 后文实验将该指标简称为延迟。⁸

⁶译注: 程序及输入/输出长度来自真实或代表性任务, 但到达轨迹是人工合成的泊松负载; 这里不是在生产流量上测量全部结果。

⁷对多线程程序, 程序级 token 延迟为关键路径响应时间除以所有线程的总生成 token 数。

⁸译注: 首 token 时间通常缩写为 TTFT, 原文的 TFFT 为字母

基线。比较三种基线。包括 Autellix 在内, 所有方案均采用相同最大批量。

- **vLLM** [36]。先进的高吞吐量服务系统, 结合连续批处理 [86] 和 PagedAttention [36], 减少 KV 缓存碎片。默认 FCFS 策略不感知应用, 会产生调用级与程序级队头阻塞。实验使用 vLLM v0.6.1。
- **vLLM-opt**。启用分块预填充 [3]、前缀缓存 [41, 90] 和多步调度的优化版本。根据 vLLM 博客 [72], 其性能接近 SGLang [90] 与 TensorRT [46]。
- **MLFQ**。在 vLLM-opt 上实现多级反馈队列抢占算法 [77], 用于区分程序级与调用级阻塞的影响。

6.3 单引擎端到端性能

图 12 比较三种基线与 Autellix, 覆盖 ShareGPT、BFCL、LATS 和混合负载。在所有负载上, 相同 token 延迟下, Autellix 都达到最高吞吐量。vLLM 表现最差, 原因是没有前缀缓存, 同一程序的后续调用需要昂贵地重新计算累计状态 (图 7)。vLLM-opt、MLFQ 和 Autellix 的相对表现则随工作负载变化。

顺序笔误。本文“延迟”是按整个程序生成 token 总量归一化的时间, 不等同于一次 LLM 调用的 TPOT, 也不等同于未经归一化的程序总完成时间; 并行线程的总 token 数仍全部计入分母。

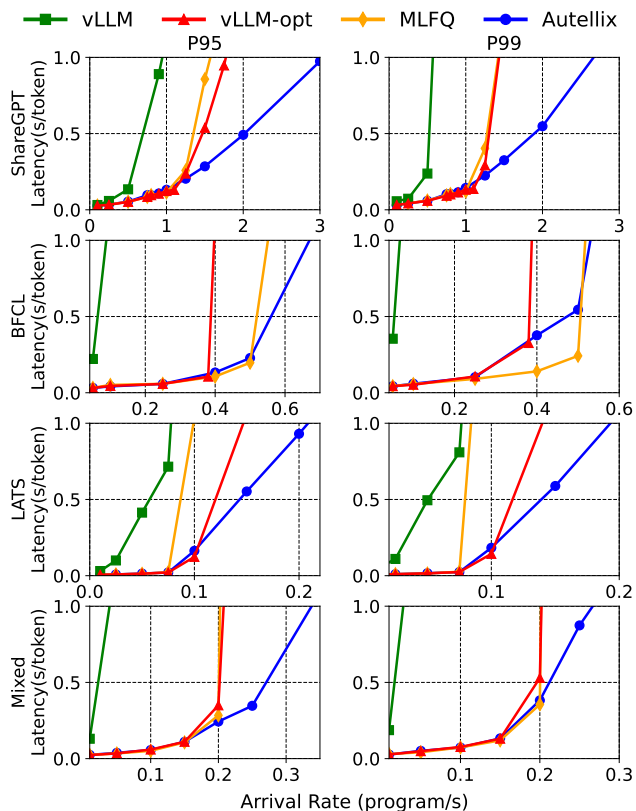


图 13: 单引擎尾延迟。不同服务系统的第 95 (P95) 与第 99 (P99) 百分位程序级 token 延迟。各行依次为 ShareGPT、BFCL、LATS 和混合负载, 左右两列分别为 P95 与 P99。横轴为程序到达率 (程序/秒), 纵轴为延迟 (秒/ token); 模型为 LLaMA-3.1-8B, 图例为 vLLM、vLLM-opt、MLFQ 和 Autellix。

前两行展示单线程负载 ShareGPT 与 BFCL。vLLM 和 vLLM-opt 的 FCFS 调度产生严重队头阻塞, 程序到达率提高时延迟增加。MLFQ 的抢占机制缓解调用级阻塞, 使吞吐量达到 vLLM-opt 的约 1.5 倍, 但高负载下仍受程序级阻塞影响。利用 PLAS 同时处理两个层面的阻塞, Autellix 在高负载下吞吐量最高达到 vLLM 的 8 倍、vLLM-opt 的 2 倍以及 MLFQ 的 1.5 倍。

第三行给出多线程 LATS 结果。Autellix 的吞吐量最高分别达到 vLLM、MLFQ 和 vLLM-opt 的 5 倍、2.5 倍、2 倍。对单线程有利的 MLFQ 抢占机制, 在多线程场景中效果较差: 过度优先执行短请求, 会扰乱同一程序各线程的推进, 加重程序级阻塞, 使整体进度停滞。ATLAS 则整体优化所有线程的资源分配, 保持较均衡的推进, 在高多线程负载下持续提供较高吞吐量。

图 12 第四行给出混合负载结果。Autellix 的吞吐量最高达到 vLLM 的 15 倍、MLFQ 的 5.5 倍和 vLLM-opt 的 4 倍。由于它同时降低程序级与调用级阻塞, 随着程序异构性, 即调用数量和解码长度的多样性增加, 收益更明显。

尾延迟。 抢占调度能降低平均延迟, 但长程序饥饿可能提高尾延迟。图 13 给出 LLaMA-3.1-8B 上各工作负载的第 95 (P95) 与第 99 (P99) 百分位延迟。ShareGPT 上,

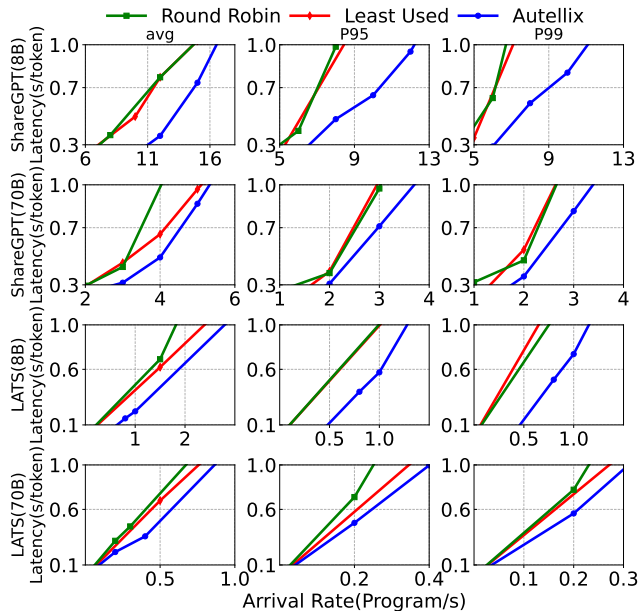


图 14: 多引擎主要结果。不同负载均衡策略的平均、P95 与 P99 延迟。各行依次为 ShareGPT / 8B、ShareGPT / 70B、LATS / 8B、LATS / 70B; 各列依次为平均、P95、P99。横轴为程序到达率 (程序/秒), 纵轴为程序级 token 延迟 (秒/ token)。Round Robin 为轮转分配, Least Used 为分给调用数最少的引擎, Autellix 为本文策略; 三种策略使用相同程序感知调度器。

MLFQ 相对 vLLM-opt 明显改善平均延迟 (图 12), 但 P95/P99 较差; BFCL 上, 它在两类指标上都优于 vLLM-opt。在八种情况中的七种, Autellix 的尾延迟稳定低于 MLFQ 和 vLLM-opt; 在给定 P95/P99 尾延迟下, 吞吐量最多提高至 1.7 倍, 体现出平均与尾部性能上的收益。

6.4 多引擎端到端性能

为评估感知数据局部性的负载均衡器 (§4.3), 我们与两种常用负载均衡策略比较。为公平起见, 它们使用相同的调度策略, 即 PLAS 和 ATLAS:

- **轮转 (Round Robin)**。按循环顺序分配请求, 使各引擎请求数均匀; 原文将其描述为 Kubernetes 的默认负载均衡策略 [33]。它忽略数据局部性, 因此产生较多 KV 缓存未命中与重新计算开销。
- **最少使用 (Least Used)**。把请求分到系统内 LLM 调用数最少的引擎, 有效平衡负载。但与轮转一样, 它忽略局部性, 导致频繁重新计算 KV 缓存。

我们在四个 LLaMA-3.1-8B 副本与两个 LLaMA-3.1-70B 副本配置上, 分别使用 ShareGPT 和 LATS 进行实验。图 14 表明, Autellix 在所有配置下都能保持较低平均延迟与尾延迟, 吞吐量最多达到两种基线的 1.4 倍。ShareGPT 的收益更明显, 因为复用聊天历史能显著增强 KV 缓存局部性。副本增加时, 这些优势更突出: 引擎数量越多, 不考虑局部性的路由越不容易选中已经保有相关缓存的引擎。

可扩展性。 我们使用 LLaMA-3.1-8B 上的 ShareGPT,

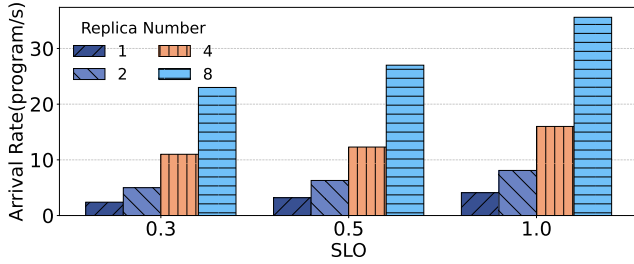


图 15: **可扩展性实验**。相同 SLO, 即以秒 / token 定义的程序级延迟目标下, Autellix 的最大程序到达率随副本数, 即 LLM 引擎数, 近似线性增加。横轴 SLO 为延迟目标, 纵轴为最大到达率 (程序 / 秒); Replica Number 图例为 1、2、4、8 个副本。

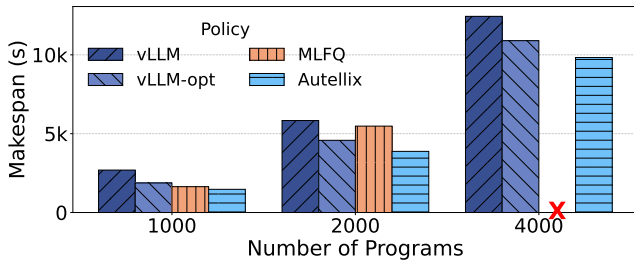


图 16: **离线批量推理**。Autellix 缩短处理一批程序所需的总时间, 即完成跨度。横轴 Number of Programs 为程序数, 纵轴 Makespan 为完成跨度 (秒); Policy 图例比较 vLLM、vLLM-opt、MLFQ 和 Autellix。4000 个程序时 MLFQ 的红叉表示内存不足而未能完成, 不能将该点当作零耗时。

考察不同延迟要求下, 增加引擎副本数时的性能。图 15 中, 各设置都呈线性扩展。利用程序级负载均衡, Autellix 有效地横向扩展, 避免路由造成的数据局部性损失, 适用于大规模部署。

6.5 消融实验

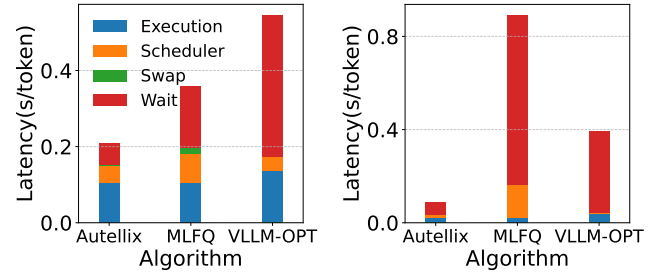
我们在不同场景中进行消融, 包括离线批量推理、时间构成, 以及交换内核等设计选择。所有实验均使用 LLaMA-3.1-8B [24], 运行 ShareGPT [1] 和 LATS [91]。

6.5.1 离线推理

离线场景更重视吞吐量而非交互延迟, 会批量处理大量程序。我们考察所有程序在开始时一次提交的情形。图 16 给出 ShareGPT 上, 各系统处理全部程序的完成跨度。Autellix 稳定优于基线, 平均完成跨度缩短 10%–40%。程序数为 4000 时, MLFQ 未能完成: 它把所有新请求分到最高优先级队列, 形成大量活动调用, 造成严重内存争用与频繁 GPU–CPU 交换, 最终资源耗尽; 即使交换空间超过 1.2 TB, 也仍发生内存不足 (OOM)。

6.5.2 时间构成

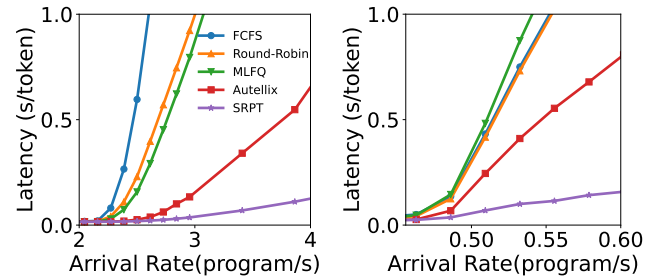
图 17 分解各方案 LLM 调用在服务层花费的时间。Autellix 在 ShareGPT 与 LATS 上都取得较低 token 延迟: 程序级调度减少等待, 改进交换内核减少交换时间。抢占的调度成本较高, 因此 Autellix 和 MLFQ 的调度时间都高于 vLLM-opt 的简单 FCFS。但 Autellix



(a) ShareGPT

(b) LATS

图 17: **推理时间构成**。Autellix 显著缩短等待时间, 相对 vLLM 只增加较小调度开销, 并通过改进内核缩短交换时间。横轴为算法, 纵轴为按 token 归一化的延迟 (秒 / token); Execution、Scheduler、Swap、Wait 分别为实际执行、调度、GPU–CPU 交换和等待。左右图分别为 ShareGPT 与 LATS。



(a) ShareGPT

(b) LATS

图 18: **与最优调度策略比较**。模拟中, Autellix 优于其他非预知调度, 但相对已知运行时长的 SRPT 仍有明显差距。左右图分别为 ShareGPT 与 LATS; 横轴为程序到达率 (程序 / 秒), 纵轴为程序级 token 延迟 (秒 / token)。图例比较先来先服务 FCFS、轮转 Round-Robin、多级反馈队列 MLFQ、Autellix 与最短剩余处理时间 SRPT。最优性讨论限于正文所述的简化模拟设定。

利用程序级优先级, 将调用更合理地分散到不同优先级队列, 调度开销仍低于 MLFQ。后者默认将新调用放入最高优先级队列, 大部分调用集中在高优先级队列, 导致较大调度开销。

6.5.3 与最优调度比较

SRPT 等最优调度策略假设完整知道各程序运行时长, 这在实践中并不现实。因此, 我们在模拟器中预先公开每个程序的总调用次数与解码步数, 模拟可预知工作负载的条件。模拟只考虑调度, 并假设各连续批处理步骤耗时相同。在这些简化条件下, Autellix 优于 FCFS 和轮转、MLFQ 等抢占调度。不过, 它与 SRPT 仍有明显差距, 说明先验知识可以显著改善性能。⁹

⁹译注: 该实验是固定步骤成本的简化模拟, 不能将 SRPT 曲线解释为实际多引擎、缓存与传输约束下动态 DAG 调度的严格全局最优界。

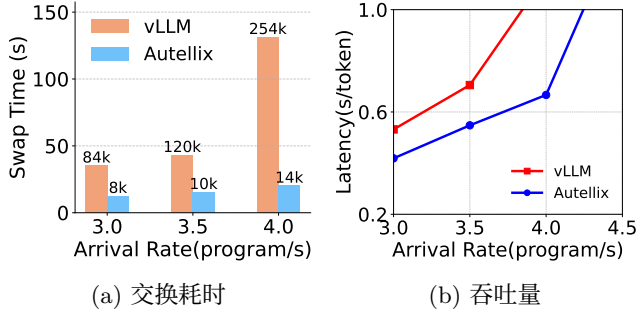


图 19: Autellix 交换内核的影响。合并传输减少交换操作次数与 GPU-CPU 交换耗时, 提高吞吐量。两图横轴均为程序到达率 (程序/秒); 左图纵轴 Swap Time 为总交换耗时 (秒), 柱上数字为操作计数, k 表示千; 右图纵轴为程序级 token 延迟 (秒/ token)。图例比较 vLLM 原有交换内核与 Autellix 改进内核。

6.5.4 交换内核的影响

抢占使系统中的活动调用增加, GPU 内存利用率较高。为获取相关 KV 缓存, 需要频繁在 GPU 与 CPU 之间交换; 高请求率下, 交换开销显著 [77]。Autellix 将并行的 KV 块传输合并为一次操作, 交换操作次数最多减少至原来的约 1/18, 交换时间减少至约 1/3-1/7; 相对 vLLM 原有内核, 吞吐量提高至 1.3 倍 (图 19)。¹⁰

7 讨论与未来工作

图优化。 Autellix 不假设预先知道程序执行 DAG, 而是在运行时动态构建其内部表示。事先完整知道程序执行过程并不现实, 但预测紧接着的下一步可能可行; 这能支持分支预测、推测执行等编译优化, 让未来调用在此前调用尚未完成时开始执行。这些优化留待未来研究。

后训练。 DeepSeek-R1 [19]、OpenAI o1/o3 [52] 等推理模型, 通过端到端强化学习 (RL) 进行后训练, 优化思考过程。为加速训练, 分布式 RL 系统交替执行分布式同策略采样与训练, 收集轨迹并更新策略梯度 [40, 61]。通过更高效的调度, Autellix 能缩短每轮 RL 批量采样的总完成跨度, 从而直接帮助分布式后训练系统。

8 结论

我们提出 Autellix, 一个面向高度动态、通用程序的分布式 LLM 服务系统。其核心创新是利用累计服务时间等程序级统计信息, 更好地设置调用优先级并安排调度, 改善程序端到端响应时间与吞吐量。我们提出面向单线程和多线程程序的两种通用调度算法, 以及感知局部性的负载均衡器, 有效缩短程序等待时间与执行时间。实验表明, 相对 vLLM 等先进系统, Autellix 在相同延迟下将程序吞吐量提高至 4-15 倍。

致谢

感谢 IBM 的 Pravein Kannan、Diana Arroyo 和 Marquita Ellis 提供有启发的讨论。感谢 Google

DeepMind 为项目提供资金, 并提供运行实验的 AI 基础设施。Sky Computing Lab 获得以下机构的捐赠支持: Accenture、AMD、Anyscale、Google、IBM、Intel、Microsoft、Mohamed Bin Zayed University of Artificial Intelligence、Samsung SDS、SAP、Uber 和 VMware。

参考文献

- [1] Sharegpt. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered, 2023.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve, 2024.
- [3] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills, 2023.
- [4] Muhammad Akhtar, Bushra Hamid, Inayat Ur-Rehman, Mamoon Humayun, Maryam Hamayun, and Hira Khurshid. An optimized shortest job first scheduling algorithm for cpu scheduling. *J. Appl. Environ. Biol. Sci.*, 5(12):42-46, 2015, 5:42-46, 01 2015.
- [5] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. Operating systems: Three easy pieces - cpu scheduling: Multi-level feedback queue. Online; accessed December 8, 2024, 2018. Available at <https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf>.
- [6] Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning, 2024.
- [7] Wei Bai, Li Chen, Kai Chen, Dongsu Han, Chen Tian, and Hao Wang. Information-Agnostic flow scheduling for commodity data centers. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 455-468, Oakland, CA, May 2015. USENIX Association.
- [8] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682-17690, March 2024.

¹⁰ 译注: 这里的次数收益来自合并逐块传输操作, 不能据此认定实际传输的 KV 字节总量也降低 18 倍; 图中的柱上数字为交换操作计数。

- [9] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024.
- [10] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners, 2020.
- [11] Sequoia Capital. Generative ai’s act i: The era of creation, 2024. Accessed: 2024-11-19.
- [12] Lingjiao Chen, Jared Quincy Davis, Boris Hanin, Peter Bailis, Ion Stoica, Matei Zaharia, and James Zou. Are more llm calls all you need? towards scaling laws of compound inference systems, 2024.
- [13] Yinlam Chow, Guy Tennenholtz, Izzeddin Gur, Vincent Zhuang, Bo Dai, Sridhar Thiagarajan, Craig Boutilier, Rishabh Agarwal, Aviral Kumar, and Aleksandra Faust. Inference-aware fine-tuning for best-of-n sampling in large language models, 2024.
- [14] Mosharaf Chowdhury and Ion Stoica. Efficient coflow scheduling without prior knowledge. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM ’15*, page 393–406, New York, NY, USA, 2015. Association for Computing Machinery.
- [15] LangChain Contributors. Langchain: Building applications with llms through composability. <https://www.langchain.com>, 2023. Accessed: November 19, 2024.
- [16] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022.
- [17] Jared Quincy Davis, Boris Hanin, Lingjiao Chen, Peter Bailis, Ion Stoica, and Matei Zaharia. Networks of networks: Complexity class principles applied to compound ai systems design, 2024.
- [18] DeepMind. Ai solves imo problems at silver medal level. <https://tinyurl.com/brmnma2a>, 2024. Accessed: 2024-11-18.
- [19] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [20] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023.
- [21] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023.
- [22] Jonathan Evans. Heuristic and analytic processes in reasoning. *British Journal of Psychology*, 75(4):451–468, 1984.
- [23] Genspark. Genspark: The ai agent engine, 2024.
- [24] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The llama 3 herd of models, 2024.
- [25] Isaac Grosz, Ziv Scully, and Mor Harchol-Balter. Srpt for multiserver systems. *Performance Evaluation*, 127-128:154–175, 2018.
- [26] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A gpu cluster manager for distributed deep learning. In *Proceedings of the 16th USENIX Conference on Networked Systems Design and Implementation, NSDI’19*, page 485–500, USA, 2019. USENIX Association.
- [27] Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis, 2024.
- [28] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023.
- [29] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [30] Daniel Kahneman. Maps of bounded rationality: Psychology for behavioral economics. *The American Economic Review*, 93(5):1449–1475, 2003.
- [31] Daniel Kahneman. *Thinking, Fast and Slow*. Allen Lane, 2011.
- [32] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri

- Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023.
- [33] Kubernetes Manual. <https://kubernetes.io/docs/home/>, 2017. [Online; accessed 04-Dec-2017].
- [34] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, Lei M Zhang, Kay McKinney, Disha Shrivastava, Cosmin Paduraru, George Tucker, Doina Precup, Feryal Behbahani, and Aleksandra Faust. Training language models to self-correct via reinforcement learning, 2024.
- [35] Adarsh Kumarappan, Mo Tiwari, Peiyang Song, Robert Joseph George, Chaowei Xiao, and Anima Anandkumar. Leanagent: Lifelong learning for formal theorem proving, 2024.
- [36] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.
- [37] LangChain Inc. Langgraph: A library for building stateful, multi-actor applications with llms. <https://github.com/langchain-ai/langgraph>, 2024. Accessed: 2024-11-18.
- [38] Kuang-Huei Lee, Ian Fischer, Yueh-Hua Wu, Dave Marwood, Shumeet Baluja, Dale Schuurmans, and Xinyun Chen. Evolving deeper llm thinking, 2025.
- [39] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Alpaserve: Statistical multiplexing with model parallelism for deep learning serving, 2023.
- [40] Eric Liang, Richard Liaw, Philipp Moritz, Robert Nishihara, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning, 2018.
- [41] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable, 2024.
- [42] Michael Luo, Justin Wong, Brandon Trabucco, Yanping Huang, Joseph E. Gonzalez, Zhifeng Chen, Ruslan Salakhutdinov, and Ion Stoica. Stylus: Automatic adapter selection for diffusion models, 2024.
- [43] Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. Aios: Llm agent operating system, 2024.
- [44] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. sl: Simple test-time scaling, 2025.
- [45] Misja Nuyens and Adam Wierman. The foreground-background queue: A survey. *Performance Evaluation*, 65(3):286–307, 2008.
- [46] NVIDIA. Nvidia tensorrt-llm. <https://github.com/NVIDIA/TensorRT-LLM>, 2023. Accessed: 2023-12-09.
- [47] OpenAI. Openai operator: Computer using agent. <https://openai.com/index/computer-using-agent/>. Accessed: 2025-02-04.
- [48] OpenAI. Gpt-4 technical report, 2023.
- [49] OpenAI. Chat Completions API. <https://platform.openai.com/docs/guides/chat-completions>, 2024. Accessed: December 08, 2024.
- [50] OpenAI. Swarm - a github repository for swarm intelligence, 2024. Accessed: 2024-12-01.
- [51] OpenAI. Chat completions. <https://platform.openai.com/docs/api-reference/chat>, 2025. Accessed: 2025-02-19.
- [52] OpenAI. Introducing OpenAI o1. <https://openai.com/o1>, 2025. Accessed on February 04, 2025.
- [53] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024.
- [54] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023.
- [55] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models, 2023.

- [56] Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. Agent q: Advanced reasoning and learning for autonomous ai agents, 2024.
- [57] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving, 2024.
- [58] Mitchell Rosser and Marc. G Carmichael. Two heads are better than one: Collaborative llm embodied agents for human-robot interaction, 2024.
- [59] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.
- [60] Dhruv Shah, Blazej Osinski, Brian Ichter, and Sergey Levine. Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action, 2022.
- [61] Guangming Sheng, Yuhan Zhang, Zixuan Xu, Yifan Jiang, Jiaao Jiang, Hao Jiang, Yong Xue, and Jinyang Li. Hybridflow: A flexible and efficient rlhf framework. 2024.
- [62] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-lora: Serving thousands of concurrent lora adapters, 2024.
- [63] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu, 2023.
- [64] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [65] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024.
- [66] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. Preble: Efficient distributed prompt scheduling for llm serving, 2024.
- [67] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Lluminix: Dynamic scheduling for large language model serving, 2024.
- [68] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Lluminix: Dynamic scheduling for large language model serving, 2024.
- [69] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. Llama: Open and efficient foundation language models, 2023.
- [70] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [71] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [72] vLLM Team. Performance update on vllm, September 2024. Accessed: 2024-12-09.
- [73] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities, 2024.
- [74] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, et al. Openhands: An open platform for ai software developers as generalist agents, 2024.
- [75] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, and Manling Li. Ragen: A general-purpose reasoning agent training framework, 2025.
- [76] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [77] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models, 2024.
- [78] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.
- [79] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and

Joseph E. Gonzalez. Berkeley function calling leaderboard. 2024.

- [80] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024.
- [81] Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E. Gonzalez, and Bin Cui. Buffer of thoughts: Thought-augmented reasoning with large language models, 2024.
- [82] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018.
- [83] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023.
- [84] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.
- [85] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.
- [86] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.
- [87] Xiao Yu, Baolin Peng, Vineeth Vajipey, Hao Cheng, Michel Galley, Jianfeng Gao, and Zhou Yu. Exact: Teaching ai agents to explore with reflective-mcts and exploratory learning, 2024.
- [88] Shuai Zhao, Xiaotian Dai, Iain Bate, Alan Burns, and Wanli Chang. Dag scheduling and analysis on multiprocessor systems: Exploitation of parallelism and dependency. In *2020 IEEE Real-Time Systems Symposium (RTSS)*, pages 128–140, 2020.
- [89] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica.

Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

- [90] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024.
- [91] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models, 2024.
- [92] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024.
- [93] Xuanhe Zhou, Xinyang Zhao, and Guoliang Li. Llm-enhanced data management, 2024.
- [94] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. Nanoflow: Towards optimal large language model serving throughput, 2024.
- [95] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Language agents as optimizable graphs, 2024.